

Webengineering

Website + JavaScript Aufgabe 2

vom 28.04.2026 bis 24.05.2026

an der Dualen Hochschule Baden-Württemberg

im Studiengang Informatik/ Informationstechnik

Fakultät:	Technik
Studienjahrgang:	2024
Studienhalbjahr:	4. Semester
Matrikelnummer:	4898576
Kurs:	INF24IN
Name des Betreuers:	Jürgen Schneider

Erfahrungsbericht

Gut:

Ich persönlich war sehr überrascht, wie einfach und intuitiv React zu benutzen war. Es hat sich sehr stimmig angefühlt und war durch seine vielen eingebauten Funktionen sehr dynamisch. Ebenfalls das Verstehen und Nutzen von APIs sowie deren ansprechende Darstellung hat eine interessante Herausforderung dargestellt — das fertige Produkt war am Ende schön anzusehen. Sich Gedanken zu machen, wie Elemente eingeblendet werden, welche wann angezeigt oder überdeckt werden und das Ganze mit Logik zu verbinden (wie beispielsweise beim Login), fand ich persönlich sehr spannend. Auch die Verwendung von Cookies war überraschend einfach und gut nachvollziehbar. Positiv überrascht hat mich zudem, wie ausführlich externe Hilfswebseiten zu diesem Thema sind. Besonders Stack Overflow, aber auch W3Schools, waren bei manchen Debug-Sessions sehr hilfreich und regten gelegentlich zu Gedanken für weitere Komponenten an.

Schlecht:

Sehr gestört hat mich der Umgang von IDEs mit Fehlern. Meistens wurden diese nicht direkt angezeigt, sondern erschienen erst beim Laden der Seite als oft kryptische Meldung in der Konsole, was teilweise zu längeren Debug-Sessions führte. Hinzu kam, dass ein einzelner Fehler häufig die gesamte Seite zum Absturz brachte, was die Fehlersuche zusätzlich erschwerte. Auch API-Dokumentationen sind oft sehr technisch und vage formuliert, sodass man sehr viel ausprobieren musste.

Was gelernt:

Als jemand, der in der Schule bereits mit statischem HTML gearbeitet hat und in der Firma erste Berührungspunkte mit React und Java Spring hatte, war es trotzdem sehr lehrreich, ein eigenes Projekt von Grund auf aufzusetzen und aktiv auszuprobieren, was passiert, wenn man Parameter anpasst und die Seite neu lädt. Das Konzept von States und useEffect war mir vorher kaum bekannt — mittlerweile möchte ich nicht mehr darauf verzichten. Auch der Einsatz von inneren Funktionen, Event Listenern und das Denken in Komponenten war zunächst ungewohnt, hat aber sehr gut funktioniert und führte zu einem deutlich strukturierteren Programmierstil als zuvor.

JS-Aufgabe 2

Integriert in der Website (unter Browser, bzw im Browser unter dem Home-icon)

<http://localhost:4011/Excercise/webex1.htm> alternativ (wenn mit Docker gestartet)

Zu finden im Verzeichnis /public/Excercise/webex1.htm

Bearbeitete Elemente

Anzeige Wetter Daten	(1 Punkt)
Anzeige von mehreren Nachrichten als RSS Feed	(1 Punkt)
Anzeige der geplanten Züge einer auszuwählenden Bahnhof Station	(max 3 Punkte)
Gelesene Wikipedia Daten per Click vorgelesen	(1 Punkt)
Benutzer Login mit Einsatz von Cookies (ein Password für alle Nutzer)	(1 Punkt)
Benutzung von React	(3 Punkte)
Installation der Komponenten läuft über Docker	(1 Punkt)

Anmerkung

Bei der Darstellung der Züge wurde die VVS API (https://www3.vvs.de/mnq-vvs/XML_DM_REQUEST?outputFormat=rapidJSON) und die bereitgestellten Fahrpläne verwendet und sich auf das Stuttgarter Stadtbahnnetz beschränkt.

Readme

Voraussetzungen

- Docker oder npm
- Port 4011 (docker) (oder 5173 (vite)) und 4012 (Node.js) Frei
- Beliebiger, "Moderner" Browser

(Node.js, wird für den Proxy der Aufgabe benötigt)

Schritt für Schritt Installation (Docker)

1. Per Terminal in Ordner navigieren (cd ../4898576)
2. In Konsole: 'docker compose up --build'
3. Im Browser <http://localhost:4011/> aufrufen

Schritt für Schritt Installation (npm + vite)

1. Per Terminal in Ordner navigieren (cd ../4898576)
2. In Konsole:
 - a. „npm install“
 - b. „npm run dev“
3. Im Browser die angezeigte Adresse aufrufen (idr. <http://localhost:5173>)

Anmeldung mit beliebigen Nutzernamen und Passwort:

- „guest“ für poor-user
- „admin123“ für rich-user

Anmerkung: mit Docker wird ebenfalls node.js für die JavaScript Übung hochgefahren, bei vite muss dies separat erfolgen (via 'cd ../4898576/public\Excercise', 'node myWebServer.js')

Modulbeschreibung

Client-seitig:

App.jsx	Main-Komponente der React-Anwendung. Verwaltet alle States (Login, Dark-Mode, offene Fenster, Wetter), rendert die Taskleiste und koordiniert die Unter-Apps.
apps/AppStyles.js	Stylesheet. Gibt Tailwind-Klassen als JavaScript Objekte zurück, abhängig von DarkMode und weiteren States.
apps/WikiApp.jsx	Wikipedia-Suchfenster. Ruft die Wikipedia-API auf, zeit Artikel und mit Text-to-Speech API vorlesbar.
apps/SettingsApp.jsx	Einstellungsfenster. Steuert den DarkMode und Helligkeit in Apps.jsx.
Apps/BrowserApp.jsx	Eingebetteter Browser als iframe. Fängt redirects ab und zeigt Seiten inline an.
Apps/RSSFeed.jsx	Nachrichtenleser. Lädt Feeds von Tagesschau (JSON-API), SWR, Spiegel und Zeit (rss2json CORS-Proxy).
Background_Stadtbahn/index.js	Kernlogik der Stadtbahn-Simulation. Lädt Fahrplandaten, berechnet Zugpositionen und interpoliert zwischen Stationen, zeichnet sie auf einen Canvas und ruft die VVS-API für Echtzeit Delays ab. Handled mit API ebenfalls die Klicks auf Stationen und suchleiste.
Background_Stadtbahn/stylesheets.css	Styling des Hintergrunds (Karte, Popups, Labels, DarkMode).
Background_Stadtbahn/stations/*.json	Stationsdaten pro Linie mit Koordinaten, StopIDs (für API) und Fahrzeiten.
Background_Stadtbahn/schedule/*.json	Fahrplandaten aufgeteilt nach Werktag, Samstag und Sonntag/Feiertag.
Background_Stadtbahn/labels/*.png	PNG-Beschriftungen der Stationen die als Overlay auf die Karte gelegt werden.

Anmerkung:

Bei Background_Stadtbahn:

Daten wie Koordinaten von Labels wurden per Python Skript aus von VVS bereitgestellten .pdf extrahiert. Koordinaten von Stationen wurden mit weiterem Tool händisch eingetragen

Externe APIs (client-seitig aufgerufen, kein Server)

www3.vvs.de/mngvvs/XML_DM_REQUEST	Echtzeit-daten VVS
Api.open-meteo.com	Wetterdaten
Geocoding-api.open-meteo.com	Stadtname zu Koordinaten
de.wikipedia.org/w/api.php	Wikipedia-Artikel
Tagesschau.de/api2u	Nachrichtenartikel
api.rss2json.com	RSS-Feeds (CORS-Proxy für SWR/ Spiegel/ Zeit)
Google.com/recaptcha	Login-Verifikation (v2)

Server-seitig (Deployment)

Nginx.conf	Webserver-Konfiguration. Liefert die gebauten statischen React-Dateien aus und leitet alle Routen auf index.html weiter.
Dockerfile	Stage 1 baut die React-App mit Node.js, Stage 2 kopiert das Ergebnis in ein nginx-Image.
Dockerfile.node	Handled das hochfahren des Node.js servers
Docker-compose.yml	Erstellt Container, mappt Port 4011 und lädt die .env – Datei mit den API-Keys